SYSTEM DESIGN

Consensus: Paxos & Raft Visual Guide

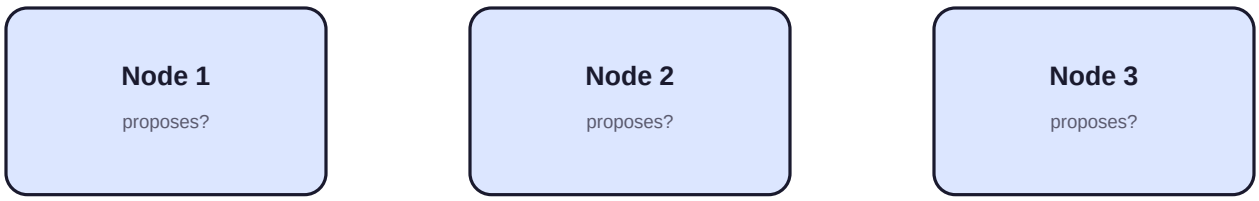
Getting unreliable machines to agree:
elections, logs, and quorums.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

The Agreement Problem

N machines, some may crash, messages may delay: still pick ONE value.



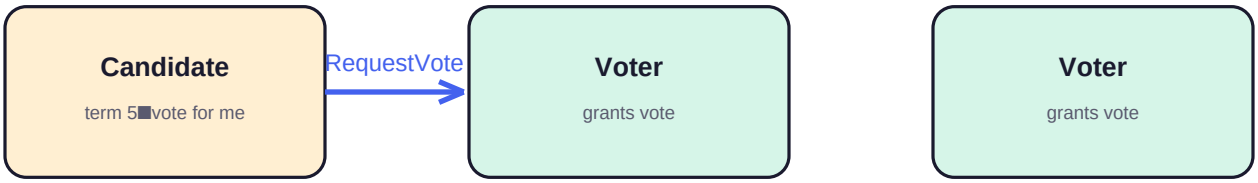
Without consensus: split-brain -- two 'leaders', divergent data, silent corruption.

- FLP impossibility: in a fully async network, no deterministic algorithm guarantees consensus -- so real systems add
- timeouts/randomization (Raft) or stable leadership to make agreement practical.
- Safety (never disagree) is non-negotiable; liveness (eventually decide) is best-effort.

STEP 2/6

Raft: Leader Election

Terms, votes, and the rule that only one leader wins a term.

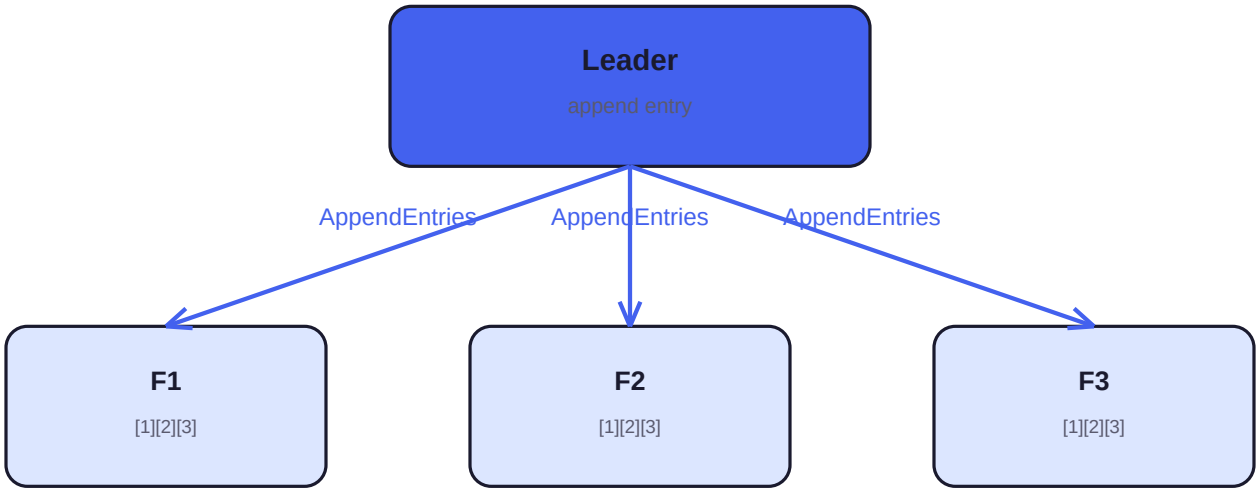


- Election timeout (randomized 150-300ms): followers who hear nothing become candidates -- randomization prevents ties.
- One vote per node per term; majority wins. A new leader's first act: establish authority (no-op log entry).
- Leaders send heartbeats; a partitioned old leader steps down when it sees a higher term.

STEP 3/6

Raft: Log Replication

The leader's log is truth; followers copy it entry by entry.

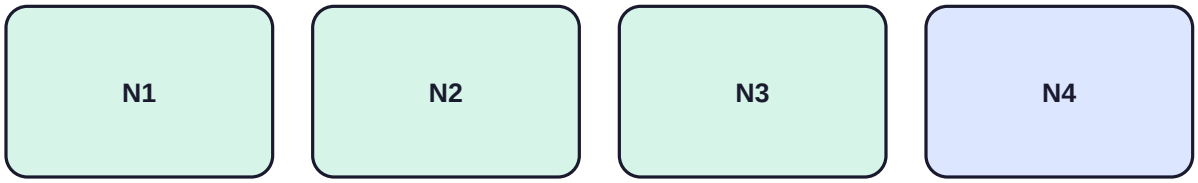


- Entries carry (term, index): followers reject appends that do not match their log prefix -- logs stay identical.
- Commit rule: an entry is COMMITTED once a majority has it; then the leader applies it and tells followers.
- Leader completeness: any elected leader holds all committed entries -- committed data is never lost.

STEP 4/6

Quorums: Majorities Decide

With 2f+1 nodes, any two majorities overlap -- that overlap is the safety.



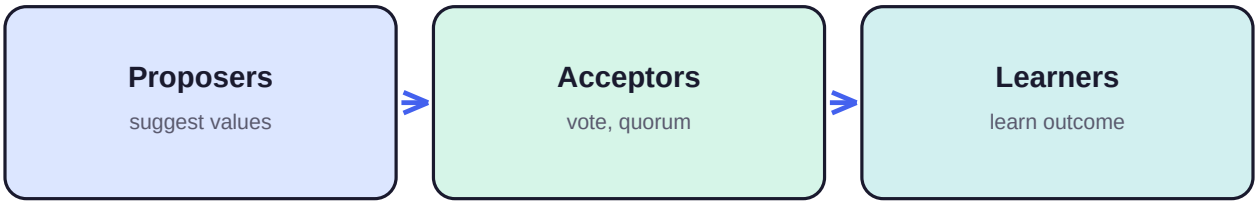
5 nodes tolerate 2 failures. Write needs 3 acks; election needs 3 votes.

- Majority intersection: no two majorities can disagree, so no two leaders commit conflicting entries.
- Even node counts are wasteful: 4 nodes tolerate 1 failure, same as 3 -- always use odd cluster sizes.
- Witness/arbiter nodes vote without storing data -- cheap quorum members.

STEP 5/6

Paxos: Roles and Phases

The classic: proposers, acceptors, learners -- two phases to agree on one value.



Phase 1 (prepare): proposer asks acceptors to promise. Phase 2 (accept): proposer sends the value.

- Promise rule: acceptors ignore lower-numbered proposals -- the highest proposal always wins cleanly.
- Multi-Paxos: elect a stable leader once, then skip phase 1 -- this is what Raft essentially streamlined.
- Paxos is the theory; Raft is Paxos made teachable. Same safety, friendlier story.

STEP 6/6

Where Consensus Lives

You rarely implement it -- you operate systems built on it.

etcd

Raft; Kubernetes' brain -- every cluster state change is a consensus round

ZooKeeper

ZAB (Paxos-like); config, locks, leader election

CockroachDB / TiKV

Raft per range/shard -- consensus at data scale

Blockchains

BFT consensus -- agreement with ADVERSARIES, not just crashes

STEP ✓I✓

Key Takeaways

Consensus: Paxos & Raft — Remember

- Consensus lets crash-prone machines agree on one value; safety (never disagree) beats liveness.
- Raft elects a leader per term by majority vote; randomized timeouts prevent election ties.
- The leader replicates its log; an entry commits once a majority holds it -- committed = durable.
- Majority quorums intersect, so conflicting decisions cannot both commit; use odd cluster sizes.
- Paxos (proposers/acceptors/learners) is the theory; Raft is the operable telling; etcd/ZooKeeper run your world on it.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev